K近邻分类练习题

在这个练习中，我们使用电信企业的客户流失数据集，Orange_Telecom_Churn_Data.csv（存放在当前目录下）。我们先读入数据集，做一些数据预处理，然后使用K近邻模型根据用户的特点来预测其是否会流失。

第一步：

- 将数据集读入变量data中，并查看其前5行。
- 去除其中的“state”，“area_code”和“phone_number”三列。

在 [18]…

```
# 将数据集读入变量data中，并查看其前5行
import pandas as pd
data = pd.read_csv('Orange_Telecom_Churn_Data.csv')
print(data.head(5))
```

```
  state  account_length  area_code phone_number intl_plan voice_mail_plan  \
0    KS             128        415     382-4657        no             yes
1    OH             107        415     371-7191        no             yes
2    NJ             137        415     358-1921        no              no
3    OH              84        408     375-9999       yes              no
4    OK              75        415     330-6626       yes              no

   number_vmail_messages  total_day_minutes  total_day_calls  \
0                     25              265.1              110
1                     26              161.6              123
2                      0              243.4              114
3                      0              299.4               71
4                      0              166.7              113

   total_day_charge  ...  total_eve_calls  total_eve_charge  \
0             45.07  ...               99             16.78
1             27.47  ...              103             16.62
2             41.38  ...              110             10.30
3             50.90  ...               88              5.26
4             28.34  ...              122             12.61

   total_night_minutes  total_night_calls  total_night_charge  \
0                244.7                 91               11.01
1                254.4                103               11.45
2                162.6                104                7.32
3                196.9                 89                8.86
4                186.9                121                8.41

   total_intl_minutes  total_intl_calls  total_intl_charge  \
0                10.0                 3               2.70
1                13.7                 3               3.70
2                12.2                 5               3.29
3                 6.6                 7               1.78
4                10.1                 3               2.73

   number_customer_service_calls  churned
0                              1    False
1                              1    False
2                              0    False
3                              2    False
4                              3    False

[5 rows x 21 columns]
```

在 [19]…

```python
# 去除“state", "area_code"和"phone_number"三列
data = data.drop(columns=['state', 'area_code', 'phone_number'])
```

在 [20]…

```python
print(data.columns)
```

```
Index(['account_length', 'intl_plan', 'voice_mail_plan',
       'number_vmail_messages', 'total_day_minutes', 'total_day_calls',
       'total_day_charge', 'total_eve_minutes', 'total_eve_calls',
       'total_eve_charge', 'total_night_minutes', 'total_night_calls',
       'total_night_charge', 'total_intl_minutes', 'total_intl_calls',
       'total_intl_charge', 'number_customer_service_calls', 'churned'],
      dtype='object')
```

第二步:

- 有些列的值是分类数据，如'intl_plan'， 'voice_mail_plan'， 'churned'这三列，需要把它们转换成数值数据。

在 [21]…

```
from sklearn.preprocessing import LabelBinarizer

lb = LabelBinarizer()

for col in ['intl_plan', 'voice_mail_plan', 'churned']:
    data[col] = lb.fit_transform(data[col])
data.head(5)
```

出[21]:

	account_length	intl_plan	voice_mail_plan	number_vmail_messages	total_day_m
0	128	0	1	25	
1	107	0	1	26	
2	137	0	0	0	
3	84	1	0	0	
4	75	1	0	0	

第三步：

- 将除“churned”列之外的所有其他列的数据与“churned”列的数据分开，即创建两张数据表，X_data和y_data。
- 使用课件中提到的某种尺度转换方法（scaling method）来缩放X_data。

在 [22]…

```
# 生成X_data和y_data
X_data = data.drop(columns=['churned'])
y_data = data['churned']
```

在 [23]…

```
from sklearn.preprocessing import StandardScaler  # 补充导入语句
```

在 [24]…

```
# 缩放X_data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_data)
```

第四步：

- 创建一个k=3的K近邻模型，并拟合X_data和y_data。

在 [25]…

```
# 创建一个3NN模型，并训练
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_scaled, y_data)
```

输出[25]…

▾ KNeighbors分类器

```
KNeighborsClassifier(n_neighbors=3)
```

第五步：

- 用上一步训练好的K近邻模型预测相同的数据集，即X_data，并评测预测结果的精度。

在 [26]…

```
# 预测并评价
from sklearn.metrics import accuracy_score
y_pred = knn.predict(X_scaled)
accuracy = accuracy_score(y_data, y_pred)
print(f"\n模型精度（k=3）：{accuracy:.2f}")
```

```
模型精度（k=3）： 0.94
```

第六步：

- 构建一个同样是的模型，但是用距离作为聚集K个近邻预测结果的权重。同样计算此模型在X_data上的预测精度。 `n_neighbors=3`
- 构建另一个K近邻模型：使用均匀分布的权重，但是将闵科夫斯基距离中的指数参数设为1()，即使用曼哈顿距离。 `p=1`

在 [27]…

```
# n_neighbors=3, weights='distance'
knn_weighted = KNeighborsClassifier(n_neighbors=3, weights='distance')
knn_weighted.fit(X_scaled, y_data)
accuracy_weighted = accuracy_score(y_data, knn_weighted.predict(X_scaled))
print(f"加权投票精度：{accuracy_weighted:.2f}")
```

```
加权投票精度：1.00
```

在 [28]…

```
# n_neighbors=3, p=1
knn_manhattan = KNeighborsClassifier(n_neighbors=3, p=1)
knn_manhattan.fit(X_scaled, y_data)
accuracy_manhattan = accuracy_score(y_data, knn_manhattan.predict(X_scaled))
print(f"曼哈顿距离精度：{accuracy_manhattan:.2f}")
```

```
曼哈顿距离精度：0.94
```

第七步：

- 将K值从1变化到20，训练20个不同的K近邻模型。权重使用均匀分布的权重（缺省的）。闵科夫斯基距离的指数参数()可以设为1或者2（只要一致即可）。将每个模型得到的精度和其值存到一个列表或字典中。 `p k`
- 将和的关系绘成图表。当时，你观察到了什么?为什么? `accuracy k k=1`

在 [29]…

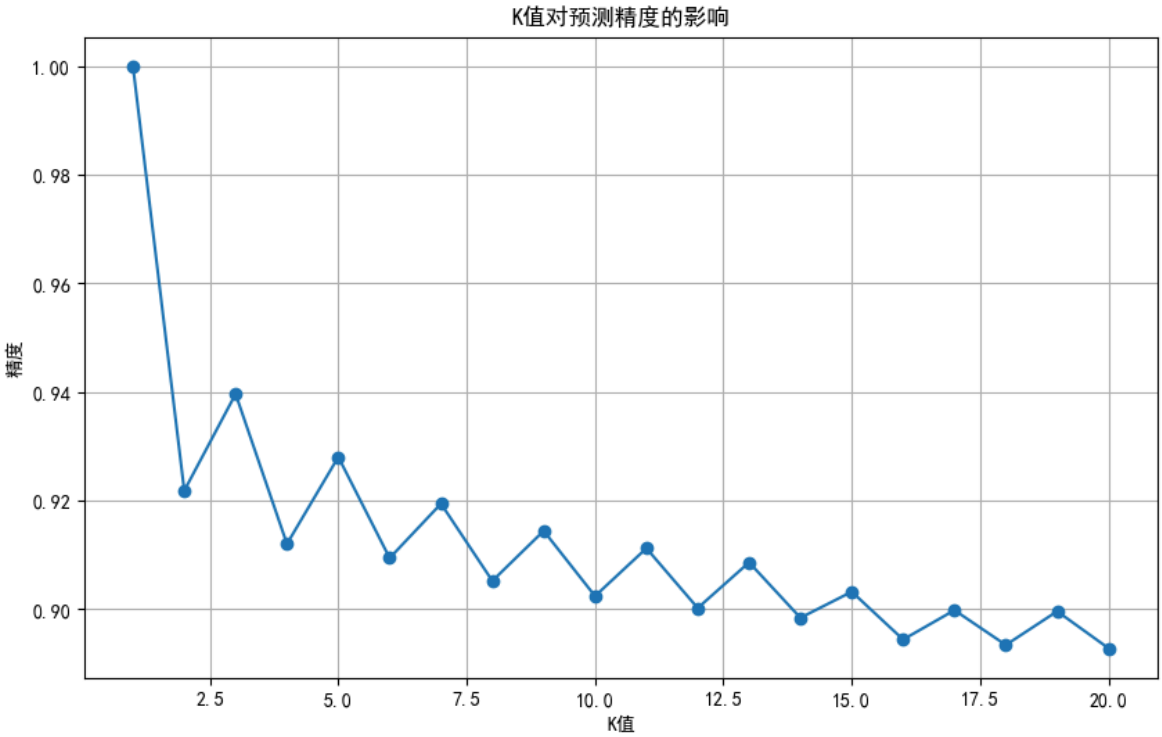
```
import matplotlib.pyplot as plt
# 测试不同K值（1-20）的精度
k_values = range(1, 21)
accuracies = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_scaled, y_data)
    accuracies.append(knn.score(X_scaled, y_data))
```

在 [30]…

```
plt.figure(figsize=(10, 6))
plt.plot(k_values, accuracies, marker='o')
plt.xlabel('K值')
plt.ylabel('精度')
plt.title('K值对预测精度的影响')
plt.grid(True)
plt.show()

print("\n观察：当k=1时精度可能最高，但容易过拟合（受噪声影响大）")
```




```
观察：当k=1时精度可能最高，但容易过拟合（受噪声影响大）
```

在 [31]…

```
import matplotlib
print(matplotlib.matplotlib_fname())
```

```
D:\anaconda3\Lib\site-packages\matplotlib\mpl-data\matplotlibrc
```

在 [32]…

```
import matplotlib.pyplot as plt
print(plt.rcParams['font.family'])
print(plt.rcParams['axes.unicode_minus'])
```

```
['SimHei']
False
```

在 [33]…

```
import matplotlib
print(matplotlib.matplotlib_fname())
```

```
D:\anaconda3\Lib\site-packages\matplotlib\mpl-data\matplotlibrc
```

在 [34]…

```
import matplotlib
print(matplotlib.get_cachedir())
```

```
C:\Users\Lenovo\.matplotlib
```